Wit — Language Specification

Write everything. Render anything.

v0.1.0
June 2026

Introduction

Wit is a plain-text markup language with the formatting power of HTML and the composability of a component framework, designed around a single principle: the writer should never have to take off the writing hat.

You write prose. The structure, the formatting, the citations, the figures, the branching, the data — all of it lives in a syntax light enough that it never interrupts the sentence you're in the middle of. And because a Wit document parses into a clean tree — the same kind of structure HTML produces — anything downstream can consume it: a renderer, a typesetter, a game engine, a static-site builder, a data pipeline.

It is one source format for books, theses, academic articles, blog posts, reports, technical documentation, and branching interactive scripts. The writer writes once. The renderer decides what it becomes.

This document specifies the Wit language as of v0.1.0. It supersedes the June 2026 walkthrough, picking up the surface added by milestones M13 through M17 — record-arg calls, form-fill bodies, colon-separated parameters, quoted strings, body-scatter, multi-line values, and block-aware capture substitution.

Why Wit exists

Markdown is wonderful until the moment your prose stops being filler around code and becomes the thing itself. Then it turns on you.

Markdown's default is markup unless escaped. Every line you type is treated as potential structure until proven otherwise. For a README that's fine — the prose is incidental. For a novel, an essay, or a paper, it's a quiet disaster:

- A sentence beginning "1970. It was a cold winter." becomes a numbered list, and your first word vanishes into a bullet.
- A line beginning "> half of them never wrote back" becomes a blockquote, because you happened to start with "greater than."
- Your line breaks evaporate, because single newlines collapse to spaces, and the only way to keep one is two invisible trailing spaces that editors strip.
- "I multiplied 5*6*7" renders with the 6 in italics.
- A short line above a `---` divider silently becomes a giant heading.
- The moment you need a callout, an aside, or a figure with a caption, there's no syntax — so you drop out of your manuscript and start typing raw `<figure><figcaption>` HTML.
- References are the deepest cut: you number footnotes by hand, you maintain a bibliography linked to nothing, and "see Figure 3" is a magic constant that becomes a lie the moment you

reorder anything.

Every one of those failures is the same bug wearing a different hat: markdown reads as control characters the things that are, in prose, just content — the first character of a line, ordinary punctuation, whitespace. And the betrayal is invisible in the source. Everything looks correct while you type it; you find out it broke only when you render.

Wit inverts the default. Prose unless marked. A blank line is a paragraph. A sentence is a sentence. Nothing you type means anything special until you deliberately, visibly say so. The writer stays in flow. The structure is opt-in, never ambient.

Core ideas

1. **Two audiences, one file.** A writer opens a Wit file and sees prose with light annotations. A programmer opens the same file and sees a component tree with data, control flow, and scripting. Neither is wrong. Neither gets in the other's way. The base language is complete for someone who will never write a single piece of logic — and the moment a programmer needs power, it's there, off to the side, where the prose stays clean.
2. `@` **uses,** `#` **defines.** One rule, no exceptions. You use a node with `@`. You define one with `#`. Everything in the language is one of those two gestures.
3. **No fixed vocabulary.** HTML gives you a finite set of tags decided decades ago — `<div>`, `<p>`, `<section>` — and the instant you need something it didn't anticipate, you're reaching for `class="..."` and hoping the meaning lands somewhere. Wit ships a 47-name core vocabulary that mirrors HTML's semantic shapes (headings, lists, tables, sectioning), but the language has no built-in vocabulary you're stuck with. A novelist defines `@chapter`, `@thought`, `@letter`. An academic defines `@theorem`, `@finding`, `@proof`. A game writer defines `@keeper`, `@player`, `@choice`. The node name is the meaning, and the renderer decides what each name becomes.
4. **It's a DOM.** A Wit document parses into a tree, exactly like HTML or XML. Node names carry semantics, nesting carries hierarchy, parameters carry attributes. Downstream systems walk that tree the way they'd walk a DOM — which means consuming a Wit document is a solved problem, not a parsing adventure.
5. **The writer expresses intent. The renderer supplies meaning.** This is the whole thesis. Wit is a semantic intention layer that sits between human writing and whatever system needs it. You say what you mean. The renderer decides what that means in context — a book, a website, a slide deck, a dialogue tree.

Quick taste

```
~ A small Wit file. Everything is opt-in.

#post
  title: Why I stopped using a citation manager
  author: Aldous Vane
  date: 2024-09-18
post#

@article |title @post.title|

@h1 @post.title h1@

For three years I kept my references in a database. Every paper I
read, I logged — author, title, year, the lot. And every time I sat
down to actually _write_, I broke flow to hunt down a citation key,
paste it in, and lose my thread entirely.

@callout |tone aside|
The tool meant to help me write was the thing stopping me from
writing.
callout@

article@
```

Plain prose by default. Inline emphasis with `_` / `*`. Nodes opened with `@name` and closed with `name@`. Definitions opened with `#` and closed with `#`. Parameters in pipes. A flag flipped from `draft` to `final` changes what renders, but the prose never moves.

Lexical structure

Characters

Wit source is UTF-8. The parser is byte-oriented for the special characters that drive structure (`@`, `#`, `|`, `{`, `}`, `[`, `]`, `(`, `)`, `:`, `~`, `\`, `"`, `_`, `*`, `<`, `%`) and treats everything else as prose content.

Newlines

`LF` (`\n`), `CRLF` (`\r\n`), and bare `CR` (`\r`) are all accepted as line terminators. The parser normalises to `LF` internally. A trailing newline at end-of-file is optional.

Blank lines

A blank line — a line containing only whitespace — separates paragraphs. Two or more consecutive blank lines collapse to one paragraph break.

Whitespace

Indentation is cosmetic at the top level. Scope comes entirely from explicit open/close tokens, never from whitespace. Inside form-fill bodies (see below), indentation does carry semantics for multi-line values, but the rule is local and explicit.

Tabs are treated as horizontal whitespace and may be mixed with spaces. The parser does not impose an indent width.

Comments

Two forms:

- **Line comment.** A `~` followed by U+0020 (ASCII space) at the start of a line begins a line comment that runs to end-of-line. The body never renders.

  ```
  ~ remember to verify this date before publishing
  The lighthouse was commissioned in 1847.
  ```

- **Block comment.** `~~` opens a block comment, `~~/` closes it. Block comments work mid-line and may span multiple lines. A bare `~~` inside an open block is content, so writers may use it as a divider.

  ```
  The bell ~~ TODO: confirm year ~~/ rang on the hour.
  ```

A tilde attached to a non-space byte (`~5`, `~/path`, `x~y`) is plain prose. The space after a leading-of-line `~` is load-bearing: bare `~` not followed by a space at the start of a line is content. **Always write `~` (tilde + space)** at the start of every comment line.

Comments are retained in the AST as `comment` nodes (with their text payload and source location) and elided by every standard renderer.

The prose layer

Paragraphs

Prose is the default. Write naturally. A blank line separates paragraphs. Nothing is marked up.

```
The keeper had not spoken aloud in eleven days.

He had not noticed, until his own voice surprised him.
```

The parser wraps each run of prose between block boundaries in a `paragraph` block. Soft line breaks inside a paragraph are preserved as part of the text; no markdown-style “two trailing spaces” trickery is required.

Emphasis

Two inline marks, and only two:

```
This is _italic_ and this is *bold*.
```

`_underscore_` is italic — the old typewriter convention where underlining meant “set this in italics.” `*asterisk*` is bold — the heavier, louder character earns the heavier weight. These are the only inline marks in the base language, and because they wrap a token rather than living at the start of a line, ordinary punctuation in your prose can never trigger them by accident.

`5*6*7` does not render with `6` in bold because the marks require a word-character boundary on the outside and a non-whitespace boundary on the inside.

Inline aside

Block comments are legal inline:

```
She crossed the threshold ~~ Editor's note: confirm date ~~/ alone.
```

Nodes

A node is the unit of structure. It has a name, an optional set of parameters, and an optional body.

Using a node — call forms

Wit has several call forms. All of them produce the same `nodeUse` AST kind; the parameter source differs.

Bare reference

A handle with no params and no body. Inline anywhere prose is legal.

```
She crossed the @threshold alone.
```

Block form

Opens with `@name`, closes with `name@`. Everything between is the body.

```
@aside
Fresnel lenses rarely fail. Keepers do.
aside@
```

Inline block form works the same way:

```
She crossed the @highlight last threshold highlight@ alone.
```

Parens form (self-closing)

`@name(...)` is a self-closing call. Parameters live inside the parens, separated by commas. The space form uses the first whitespace inside each slot as the key/value boundary; the colon form uses `:`.

```
@cite(author Smith, year 2024)
@cite(author: Smith, year: 2024)
@badge(tone: good)
```

Parens-form calls take no body and produce no `name@` close.

Pipes form

Parameters live inside `|...|` pipes and can appear anywhere inside a node — before the body, after it, mid-sentence. The parser collects them regardless of position, so the writer places them wherever the thought lands.

```
@figure
|lamp.png|
|caption The second-order Fresnel lens|
|full width!|
figure@
```

Pipes compose with a body close (`name@`) or can stand alone on a line:

```
@cite |author Smith| |year 2024|
```

Record-arg form (self-closing)

A record literal `{ ... }` appearing immediately after the handle (with optional whitespace between handle and `{`) becomes the call-site arguments. Fields bind by name into the template's captures.

```
@reference_entry { author – Boud, year – 2001 }
```

Or across lines:

```
@reference_entry {
  author - Boud
  year - 2001
  title - Using journal writing
}
```

The matching `}` closes the call. No `name@` close.

Form-fill body

When a node body's first non-blank, non-comment line matches `<identifier>:` followed by content (or by end-of-line plus an indented block), the body is classified as **form-fill** and every line is read as `<key>: <value>`. The result behaves like a record-arg call:

```
@cite
  author: Smith
  year: 2024
cite@
```

Multi-line values are written by leaving the key's same-line value empty and deeper-indenting the continuation:

```
@card
  title: Long card
  body:
    Multi
    line
    value
card@
```

Comment lines (`~ ...`) are ignored inside form-fill bodies. Blank lines inside an indented value block are preserved.

Body-scatter

Inside a node body that is **not** form-fill (i.e., starts as prose), the parser scans for `<id>:<v>` tokens with strict zero-whitespace contract:

- `<id>` matches `[A-Za-z][A-Za-z0-9_-]*`.
- `<v>` is a bare scalar (`[A-Za-z0-9_-]+`), a quoted string `"..."`, an inline emphasis `_..._` / `*...*`, or a `@name...` use (bare reference, parens-form, or block-shape).
- Zero whitespace anywhere in the token — including between `:` and the value. `key: @v v@` (with a space after `:`) stays prose.
- The byte immediately before `<id>` must not be a word character or `\`.

Matching tokens are lifted as scattered parameters; the rest of the body remains prose.

```
@thing mode:a name:Tauraj some prose thing@
@thing key:_italic_ thing@
@thing key:@v body v@ thing@
```

Last-one-wins on duplicate names, mirroring pipe-form scatter. The backslash escape `\:` opts out of the contract (`name\:Tauraj`) and is unescaped in the residual prose.

Mixing rule

A single call site uses **one** parameter source. Pipes, parens, record-arg, form-fill, and scatter are mutually exclusive at one site. Mixing surfaces `E_MIXED_PARAM_SOURCE`.

Bare references and block-form bodies (without form-fill or scatter) carry no params.

Parameter shapes

Inside pipes and parens, three slot shapes are recognised:

Slot	Meaning
`key value`	named parameter (first whitespace splits key from value)
`key value!`	flag (trailing `!` after a positional or value-bearing slot)
`key – value`	named parameter with multi-word key (`–` separates key/value)
`key: value`	colon-separated named parameter
`value`	positional value (no key)
`multi word value!`	named-shape flag (whole-slot name with trailing `!`)

If the same parameter appears more than once, the last one wins, applied from that point forward in the flow.

Indentation is cosmetic

Scope comes entirely from the `@name` / `name@` pairs — never from whitespace. Indent for your own readability, or don't. Both parse identically.

```
@chapter
@aside Keepers do. aside@
He read the letter twice.
chapter@
```

is identical to:

```
@chapter
  @aside Keepers do. aside@
  He read the letter twice.
chapter@
```

Defining nodes

`#` defines. This is how you build the vocabulary for your document.

Block definition

`#name` opens, `name#` closes. Inside:

- `||a, b, c||` captures parameters by name (declared list).
- `::name::` interpolates a captured value into the body.
- `...` marks where the body of each invocation will render (a body slot).

```
#chapter ||number, title||
@chapterheader
::number:: — ::title::
chapterheader@
...
chapter#
```

Used like this:

```
@chapter |number I| |title The Keeper|
For eleven days the lamp had burned untended.
chapter@
```

The captured parameters flow into the definition's structure, and the body prose lands wherever `...` sits. Parameters can be threaded into child nodes — a value passed at the use site flows down into a nested node's own parameter slot.

Implicit captures

The `||a, b, c||` capture list is optional. If omitted, captures are inferred from the `::name::` occurrences in the definition body.

```
#farewell
Goodbye, ::name::.
farewell#
```

Calling `@farewell` with any param source (`@farewell {name — Tauraj}`, `@farewell name:Tauraj`, `@farewell |name Tauraj|`) binds the inferred capture.

Single-line definitions

When a definition has no body — just a value — use the colon-bang form. The `:` signals "no body"; the value runs to `!!` (which may sit on the same line or span multiple lines).

```
#year: 1923 !!
#place: Dunmore Head !!
```

Single-line definitions can still capture and interpolate parameters:

```
#cite: ||author, year|| ::author:: (::year::) !!
```

Multi-line value definitions place the value between `:` and `!!`, spanning as many lines as needed:

```
#epigraph:
The sea does not forgive forgetting.
— coastal proverb
!!
```

If the value is a pure record `{ ... }` or collection `[ ... ]` literal, the definition collapses to a `dataDef` (rather than a `nodeDef`) so downstream consumers can read it as data without unwrapping a one-block body.

Body slot

`...` inside a definition is the body slot — the place where the prose body of each invocation renders.

```
#aside:
@panel
...
panel@
aside#
```

A definition may omit the body slot entirely if it doesn't need one. A single-line definition's value may itself be a body slot if you want a container that just wraps content.

Additive partials

Prefix a definition with `+` and multiple declarations of the same name **merge** instead of overwriting.

```
+#bibliography: @weil Simone Weil, Gravity and Grace, 1952 !!
+#bibliography: @berger John Berger, Ways of Seeing, 1972 !!
+#bibliography: @arendt Hannah Arendt, The Human Condition, 1958 !!
```

The renderer sees one merged `@bibliography` containing every contribution. No file knows about the others. Order across declarations is preserved.

Additive partials are for things that accumulate — bibliographies, glossaries, indexes, figure lists, tables of contents. They are **not** for structural nodes whose bodies can't be meaningfully merged; if the shape of two partials disagrees, the resolver raises `E_PARTIAL_SHAPE_MISMATCH`.

Records

A record is a set of named values wrapped in `{ ... }`. Records appear in three positions: as the value of a single-line definition, as a record-arg to a node use, and nested inside other records or collections.

Inline records

Short records fit on one line, comma-separated. Either `=` or `:` separates keys from values.

```
#world: { location = Bag End, time = night, storm = true } !!
#point: { x: 3, y: 7 } !!
```

Block records

The opening brace sits on the definition line; the body is indented; the closing brace sits flush left. Field separators may be commas or newlines.

```
#keeper: {
  name = Aldous Vane
  years at post = 31
  lamp lit = true
} !!
```

A `#keeper` block-shape definition with form-fill body collapses to the same record. Every form-fill line must match `<identifier>: value`, so multi-word keys belong in the brace-form record literal (above), not the form-fill body:

```
#keeper
  name: Aldous Vane
  tenure: 31
  lit: true
keeper#
```

Multi-line values

A field with empty same-line value followed by deeper-indented lines treats the indented block as the value. Each continuation line must start with the field key's leading whitespace plus at least one additional space or tab. The outer-indent prefix is stripped; the inner indent beyond that prefix is preserved. Blank lines inside the block are kept. Trailing blank lines on the value are stripped.

```
#card
  title: My card
  body:
    Line one of the body.

    Line two after a paragraph break.
card#
```

Quoted strings

Values containing commas, whitespace, or special bytes use double-quoted strings. Inside the quotes, only `\"` and `\\` are recognised as escapes.

```
#point: { name: "Came, H.", year: 2024 } !!
```

Quoted strings may span newlines unchanged.

Nested records

Records nest naturally:

```
#keeper: {
  name - Aldous Vane
  posted - Dunmore Head
  history { years - 31, incidents - 2 }
} !!
```

Multi-word keys

A key containing spaces uses the `-` separator (the first `-` on the line is the key/value boundary):

```
#tenure: { years at post - 31 } !!
```

The `:` separator implies single-word keys.

Collections

`[ ... ]` holds an ordered collection of values or records. The opening bracket sits on the definition line; each element is indented beneath it.

A collection of values:

```
#themes: [ attention, perception, moral failure ] !!
```

A collection of records — the common case for tables, casts, datasets:

```
#findings: [
  { claim - Attention precedes perception, supported - true, strength - strong }
  { claim - Looking is never neutral,       supported - true, strength - strong }
  { claim - Attention is a form of labour,  supported - true, strength - moderate }
  { claim - Perception is neutral,          supported - false, strength - contested
}
] !!
```

Records inside a collection are the same `{ ... }` records as above — one syntax for records everywhere. They map directly to the JSON-shaped data a downstream renderer expects, so there is nothing to translate.

Collections may also hold raw value-blocks delimited by `! ... !` for multi-line cell content (used by `@table` with the inline-CSV form).

Data access

Dot notation reaches into records and collections.

```
@keeper.name served for @keeper.years_at_post years.
@findings.0.claim
```

The resolver fuzzy-matches keys, so `years at post`, `years_at_post`, and `yearsAtPost` resolve to the same field — the writer never has to remember exact key formatting. Numeric segments index into collections (zero-based).

Access paths are legal anywhere a bare `@x` reference is legal: body prose, parameter values, record-RHS, collection items, conditional operands, each operands. A missing field surfaces `E_MISSING_FIELD` at expansion time.

Conditionals

Control flow in Wit is declarative and references only already-defined static data. Statements are wrapped in parentheses, so logic reads as an aside, never an instruction.

```
(if @book.status is draft)
@watermark Draft — do not distribute. watermark@
(end)
```

Comparison

`is` and `equals` are exact synonyms. Comparison is strictly type-equal: `Number(99)` matches `Number(99)`, never `String("99")`. The RHS is a bareword scalar, classified by the same typing rule as record scalars (`99` → number, `true` → boolean, `null` → null, anything else → string).

```
#meter: { value - 99 } !!

(if @meter.value is 99) On target. (end)
```

Common patterns:

```
(if @flags.enabled is true) Enabled flag is on. (end)
(if @flags.optional is null) Optional value is unset. (end)
```

Existence (truthy)

A condition that is a bare reference checks for truthiness — defined, non-empty, non-`false`, non-`null`.

```
(if @lamp.flag) The lamp burned through the watch. (end)
```

Else

```
(if @author.corresponding is true)
Corresponding author: @author.email
(else)
@author.name
(end)
```

Conditions nest freely. `(end)` always closes the nearest open statement.

Iteration

```
(each @findings as finding)
@finding |claim @finding.claim| |strength @finding.strength| finding@
(end)
```

Both collections of values and collections of records are iterable. The loop walks static data — no iteration state, no mutation, no surprises — just a clean expansion of the tree.

```
#themes: [ attention, perception, failure ] !!

(each @themes as item) The watch returned to @item. (end)
```

Iteration nests freely, may contain conditionals, and may be nested inside conditionals. Iterating over a non-iterable value surfaces `E_NOT_ITERABLE`.

This is what makes a Wit document self-organising. Define your chapters as data, and both your table of contents and your chapter bodies can render from the same source. Add a record; both update. Reorder them; both follow. Nobody maintains anything by hand.

Scripting

The escape hatch for everything the declarative model can't express is plain JavaScript, in a `<% ... %>` block. Wit does not reinvent the wheel.

A script runs once, after the initial DOM is rendered. It reads the parsed tree through a small bridge object, `lh`, manipulates it, and the final tree passes to the next system.

```
<%
// read the data
const findings = lh.data.findings;
const supported = findings.filter(f => f.supported === true).length;

// inject computed content into a waiting node
lh.inject('paper-stats', `
@statrow |label Supported| |value ${supported}| statrow@
`);
%>
```

Inline script

`<% expr %>` is legal inline anywhere a prose text run is legal. It is not legal in structural positions (node names, key names, partial keys).

```
The paper has <% lh.data.paper.wordCount %> words total.
```

Script call

`@scriptCall(fn, ...args)` invokes a function defined in a script block. The function identifier is a bareword resolved against the script scope; args are positional.

```
<%
function greet(name) {
  return `hello, ${name}`;
}
%>

The system says: @scriptCall(greet, "world")
```

Run order

Multiple `<% ... %>` blocks run strictly in document order, top to bottom, single pass. No dataflow analysis, no re-runs. Scripts execute as the parser walks past them.

Content opacity

The bytes between `<%` and `%>` are opaque to the Wit parser. The terminator is the first `%>` outside of a JavaScript string, template literal, or regex literal.

Errors

A script that throws raises `E_SCRIPT_ERROR` and aborts expansion. There is no implicit recovery — scripts are author-controlled and effectful, and silent swallowing would produce output that doesn't match author intent.

The `lh` bridge

`lh` exposes the document as a manipulable tree. The core surface:

Call	What it does
`lh.data`	all `#thing:` definitions as plain JavaScript objects
`lh.query(name)`	every node of a given type, as `{ params, content }` objects
`lh.node(id)`	a single node by its `id` parameter
`lh.sort(name, fn)`	reorder all instances of a node type
`lh.inject(id, src)`	render Wit source into a node by id
`lh.set(path, value)`	update a value in an overlay (e.g., `'paper.word count'`)
`lh.prose()`	the prose text nodes, with helpers like `.wordCount()`
`lh.host.*`	host-application-specific extensions (namespaced; not part of core)

`lh.set` writes to a script-scoped overlay consulted by later `lh.data` reads. The parsed AST is not mutated — round-trip serialisation can omit overlays unless asked.

Composition

A book, a thesis, or a large report should not live in one file. Wit composes.

References

At the top of a file, `reference` pulls in the definitions, data, and nodes from another file.

```
reference ./shared/schema.wit
reference ./shared/sources.wit
reference ./chapters/one.wit
reference ./chapters/two.wit
```

A master file becomes a composition script: it assembles the pieces. Each chapter file is self-contained prose that reaches into the shared vocabulary.

References are paths relative to the current file. Missing files raise `E_MISSING_REFERENCE_FILE`; circular reference graphs raise `E_CIRCULAR_REFERENCE`.

Additive partials across files

Combined with the `+#` prefix from the definitions section, references make a project genuinely self-assembling. Each chapter file registers its own table-of-contents entry and its own sources:

```
~ chapters/three.wit

+#toc:
@tocrow |number III| |title The Politics of Looking| tocrow@
!!

+#bibliography:
@berger_power @berger! p. 86 !
!!
```

The master file has no chapter list. It just references the files and renders `@toc` and `@bibliography` — both fully populated by the chapters that registered themselves.

Add a chapter: create the file, add one `reference` line. The contents and bibliography grow automatically. Delete a chapter: remove the line, and it vanishes from both. The document reorganises itself.

Core vocabulary

Wit ships a curated core vocabulary that mirrors HTML's semantic shapes. These 47 names need no `#` definition; the resolver skips binding lookup for them, and every standard renderer ships explicit handlers.

Category	Names
Headings	`h1`, `h2`, `h3`, `h4`, `h5`, `h6`
Inline	`em`, `strong`, `code`, `u`, `s`, `sub`, `sup`, `mark`, `small`, `br`
Lists	`ul`, `ol`, `li`, `dl`, `dt`, `dd`
Links/media	`a`, `img`, `figure`, `figcaption`, `audio`, `video`
Tables	`table`, `thead`, `tbody`, `tfoot`, `tr`, `th`, `td`, `caption`
Blocks	`p`, `blockquote`, `pre`, `hr`
Sectioning	`section`, `article`, `aside`, `header`, `footer`, `nav`, `main`
Other	`cite`

In addition, `@node` is a universal opaque pass-through: `@node |type img| |src ./lamp.png|` is treated as a generic typed node by the renderer.

```
@h1 The Lamp Keeper h1@
@ul
  @li First item li@
  @li Second item li@
ul@
```

Author-defined names are checked against the core list and skipped for binding resolution; if you `#chapter ... chapter#` and `chapter` happens to not be in the core list, the resolver uses your definition.

Tables

The `@table` core node supports three authoring forms.

Inline CSV

Rows are a collection of value-collections. The optional `caption` parameter sets the table caption; an explicit `header false` flag suppresses the header row.

```
@table |rows [[Client Hours, COUN603, COUN704, TOTAL],
              [One to One,  62.75,  41.25,  104],
              [Couple,      0.75,   0,      0.75]]|
       |caption Practicum Hours|
```

Schema as array

Pair a header schema with a `rows` reference to a collection of records. Schema keys match record fields by fuzzy match.

```
#sites: [
  { name - Dunmore Head, status - operational }
  { name - Carrick Point, status - maintenance }
] !!

@table |schema [name, status]| |rows @sites|
```

Schema as record

When the header labels differ from the data keys, use a record schema (key → label):

```
@table |schema { name - Site, status - Status }|
       |rows [[Dunmore Head, operational],
              [Carrick Point, maintenance]]|
```

Multi-line cells use `! ... !` value-blocks inside the row collection. The renderer does not emit `text-align`; numeric alignment is a theme concern.

Backslash escapes

The backslash opts out of structural interpretation in contexts where a literal character would otherwise be a control byte.

Escape	Meaning
`\:`	literal `:` — suppresses body-scatter and form-fill colon contract
`\"`	literal `"` — inside quoted strings
`\\`	literal `\` — inside quoted strings
`|`	literal `|` — inside pipe-form params
`\{`	literal `{` — opts out of record-arg detection
`\}`	literal `}` — opts out of record-arg detection

After a successful escape, the backslash is stripped from the output text. Outside of these contexts, a backslash is a literal byte and renders as-is.

Type-classified scalars

In record fields, collection items, and conditional RHS positions, scalar values are classified at parse time. Anywhere else (prose, parameter values, quoted strings), text remains text.

Source	Classification	Example
`-?[0-9]+(\.[0-9]+)?`	`numberValue`	`42`, `-3.14`
Exact `true`	`booleanValue` (true)	`enabled - true`
Exact `false`	`booleanValue` (false)	`enabled - false`
Exact `null`	`nullValue`	`optional - null`
Quoted string	`stringValue` (always)	`name - "Came, H."`
Anything else	`stringValue`	`status - draft`

Boolean and null literals are matched case-sensitively and must be the whole value. `True`, `TRUE`, `false-flag` all classify as strings.

Errors

Every error carries a stable `code`, a human-readable message, and a source location. Parser errors are `WitError`; resolver/expander errors are `ResolverError` / `ExpanderError`, both subclasses of `RuntimeError`.

Parser errors

Code	When raised
`E_UNCLOSED_NODE`	a `@name ...` reaches EOF without `name@`
`E_UNCLOSED_DEFINITION`	a `#name ...` reaches EOF without `name#` or `!!`
`E_UNCLOSED_COMMENT`	a `~~` reaches EOF without `~~/`
`E_UNCLOSED_PAREN`	`(if`, `(each`, parens-form call missing matching `)`
`E_UNCLOSED_COLLECTION`	`[` without matching `]`
`E_UNCLOSED_SCRIPT`	`<%` without matching `%>`
`E_UNTERMINATED_STRING`	`"` without matching `"` in a record value
`E_MISMATCHED_CLOSE`	a `name@` whose name doesn't match the open
`E_MALFORMED_RECORD`	`{...}` content that can't be parsed as a record
`E_MALFORMED_FORM_FIELD`	a non-comment line in a form-fill body that isn't `id:value`
`E_MIXED_PARAM_SOURCE`	record-arg combined with pipes/parens at one call site

Runtime errors

Code	When raised
`E_UNRESOLVED_REFERENCE`	a `@name` use has no matching `#name` definition
`E_CIRCULAR_REFERENCE`	reference graph contains a cycle
`E_MISSING_REFERENCE_FILE`	a `reference ./path` target does not exist
`E_MISSING_FIELD`	`@x.y` where `y` is not present on `x`
`E_MISSING_RECORD_FIELD`	record-arg missing a field the template captures
`E_EXTRA_RECORD_FIELD`	record-arg carries a field the template does not declare
`E_AMBIGUOUS_RECORD_KEY`	fuzzy match resolves to two distinct keys
`E_PARTIAL_SHAPE_MISMATCH`	`+#name` partials have incompatible shapes
`E_DUPLICATE_DEFINITION`	two non-additive `#name`s in the same scope
`E_NOT_ITERABLE`	`(each @x as i)` where `x` is not a collection
`E_TYPE_MISMATCH`	comparison RHS type doesn't match LHS type
`E_EXPANSION_DEPTH_LIMIT`	template expansion exceeds the safety depth
`E_SCRIPT_ERROR`	a `<% %>` block throws a JS exception

The AST

A Wit document parses into a tree. Every node carries a `kind` discriminator and a source `loc`. The top-level `document` node contains a flat list of blocks; blocks contain blocks or inlines, inlines contain inlines.

Block kinds

Kind	Description
`document`	top-level container
`paragraph`	a run of prose terminated by a blank line
`comment`	line or block comment, retained but never rendered
`nodeUse`	`@name ... name@`, parens-form, record-arg, etc.
`nodeDef`	`#name ... name#` with body and optional captures
`dataDef`	`#name: value !!` with a value-only payload
`record`	`{ key – value, ... }`
`collection`	`[ a, b, c ]`
`ifStatement`	`(if cond) ... (else) ... (end)`
`eachStatement`	`(each @coll as item) ... (end)`
`scriptBlock`	`<% ...js... %>` block
`referenceDirective`	`reference ./path`

Inline kinds

Kind	Description
`text`	plain prose characters
`italic`	`_..._` emphasis
`bold`	`*...*` emphasis
`interpolation`	`::name::` inside a definition body
`bodySlot`	`...` inside a definition body
`scriptCall`	`@scriptCall(fn, ...)` inline function invocation

Value kinds

Kind	Description
`stringValue`	unquoted text or `"..."` quoted string
`numberValue`	integer or decimal literal
`booleanValue`	exact `true` or `false`
`nullValue`	exact `null`

Helper kinds

Kind	Description
`param`	one slot from a pipe / paren / record-arg / scatter / form-fill
`accessPath`	a dotted path (`@x.y.0.w`) used inside a `nodeUse` handle

Condition kinds

Kind	Description
`existenceCondition`	`(if @x)` — truthiness check
`comparisonCondition`	`(if @x is value)` / `(if @x equals value)`

Every kind appears in `tests/integration/feature-tour.wit`, which is the canonical "everything in one file" reference. The full AST type definitions live in `packages/parser/src/ast.ts` and are re-exported from `@witlang/parser`.

Known limitations

The v0.1.0 parser handles every shape in this specification **except** the three rough edges below. Each one is pinned by a fixture under `tests/fixtures/` so regressions can't sneak in.

- **Multi-line pipe-form values.** A pipe parameter whose value continues across a newline (`|key value\nsecond line|`) is rejected or produces a garbled AST. Workaround: use a form-fill capture (`key: value` lines inside a bodied node) or a quoted string.
- **Nested closing of same-named nodes.** Different names nest cleanly (`@chapter ... @aside ... aside@ ... chapter@`), but the same name nesting (`@chapter ... @chapter ... chapter@ ... chapter@`) is ambiguous and resolves to the innermost match only. Workaround: rename one of the nested instances.
- **Bare tilde without trailing space.** A `~` that is not followed by a space at the start of a line parses as content, not as a comment continuation. Always write `~` (tilde + space) at the start of every comment line.
- **Control-flow statements inside `#def` bodies.** A `(if ...)` or `(each ...)` statement appearing as a block child inside a `#name ... name#` definition body interferes with the definition's close-detection. Workaround: keep control flow at the document level, outside definitions, and feed the template a flag the template reads inline (`(if ::met::) ... (end)` inside an inline expression position works; standalone block placement does not in v0.1.0).

Public API surface

The Wit toolchain is published as a small set of TypeScript packages with curated `index.ts` exports. Anything not re-exported from `index.ts` is package-private.

`@witlang/parser`

The lexer + parser. Reads Wit source and produces a typed `Document` AST.

```
import { parse, WitError, type Document } from '@witlang/parser';

try {
  const doc: Document = parse(source, '/path/to/file.wit');
  // doc.kind === 'document'
} catch (err) {
  if (err instanceof WitError) {
    console.error(`${err.code} at ${err.loc.line}:${err.loc.col}: ${err.message}`);
  }
}
```

Public exports:

- `parse(source, file?)` — main entry point.
- AST types — every block / inline / value / helper kind listed above.
- `Loc`, `HasLoc`, `WitError` — locations and parser errors.
- `parseInlineFromText(text, file?)` — inline-only parse, used by the runtime expander to re-parse captured raw values at substitution time.
- `tryParseRecordFromText` / `tryParseCollectionFromText` — late-binding data scanners exposed to the runtime.

`@witlang/runtime`

Resolves cross-file references, merges additive partials, and expands templates into a fully-substituted document tree that renderers consume.

```
import { resolve, expand } from '@witlang/runtime';

const resolved = await resolve(doc, { entry, readFile });
const expanded = expand(resolved);
```

Public exports:

- `resolve(doc, opts)` / `expand(resolved)` — the two-phase pipeline.
- `ResolveOptions`, `FileReader` — host hooks for custom resolution.
- `ResolvedDocument` / `ExpandedDocument` — output shapes.

- `RuntimeError`, `ResolverError`, `ExpanderError`, `RuntimeErrorCode`, `RuntimeErrorCodeName` — typed error surface.
- `CORE_VOCAB_NAMES`, `RESERVED_OPAQUE`, `isCoreVocabName`, `isReservedNodeName` — core vocabulary helpers.

`@witlang/render-html`

Walks an expanded document and emits HTML.

```
import { renderHtml, escapeHtml } from '@witlang/render-html';

const html = renderHtml(expanded);
```

`@witlang/render-markdown`

Walks an expanded document and emits Markdown.

```
import { renderMarkdown } from '@witlang/render-markdown';

const md = renderMarkdown(expanded);
```

`@witlang/cli`

The `wit` command.

```
wit parse <file>            Parse a .wit file, print AST as JSON.
wit check <file>            Parse + resolve, report errors (exit 1).
wit build <file> [-o output.html|output.md] [--format html|md]
wit tour <file>             Print a guided summary of every AST kind.
wit --version | --help
```

The CLI has zero runtime dependencies — it composes the public packages above.

Block-aware capture substitution

A captured raw-string value (form-fill body, record field, pipe-form value) is re-parsed at substitution / data-access time using the full parser, not just inline-parsing. Block-level constructs in the captured text (`@h1 ... h1@`, lists, tables, paragraph breaks) produce real block nodes in the expanded output instead of leaking through as literal text.

```
#card_b
  title: My card
  body:
    Line one of the body.

    Line two after a paragraph break.
    @h1 heading test h1@
card_b#

@card_b.body
```

At the access site `@card_b.body`:

- The value is re-parsed.
- Both paragraphs survive as separate blocks.
- The embedded `@h1` reaches the expander as a real `nodeUse`, not literal text.

Substitution rules:

- **Block position** (a `nodeUse` on its own line, or an interpolation whose parent paragraph contains nothing else): all parsed blocks splice in at the block-level position. Any enclosing paragraph is split around them.
- **Inline position** (mid-paragraph): if the value parses to a single paragraph, inline children splice in. If it parses to multiple blocks, the enclosing paragraph lifts into a block sequence.
- **Inside emphasis** (`_x_` / `**x**` children): only inline content from the first parsed paragraph is taken; block content is dropped.
- **Empty value**: empty splice.

Worked examples

A few small documents that lean on different parts of the language but all parse to the same kind of tree.

A short report

```
#report
  title: Q3 Operations Review
  author: Mara Finch
  quarter: Q3 2024
  status: final
report#

#metrics: [
  { label - Uptime,        value - 99.2%, target - 99.5%, met - false }
  { label - Incidents,     value - 3,     target - 5,     met - true }
  { label - Response time, value - 12 min, target - 15 min, met - true }
] !!

#metric ||label, value, target||
@metriccard
@metriclabel ::label:: metriclabel@
@metricvalue ::value:: metricvalue@
@metrictarget ::target:: metrictarget@
metriccard@
metric#

@document |title @report.title|

(if @report.status is draft)
@watermark Draft — not for circulation watermark@
(end)

@h1 @report.title h1@

(each @metrics as m)
@metric { label - @m.label, value - @m.value, target - @m.target }
(end)

document@
```

Flip `status` to `draft` and the watermark reappears. Add a row to `#metrics` and the loop grows. The prose never mentions a number that the data doesn’t already hold.

A bibliography across files

```
~ chapters/one.wit

+#bibliography:
  @reference_entry
    author: "Boud, D."
    year: 2001
    title: Using journal writing
    publisher: "New Directions, 9–18"
  reference_entry@
!!

~ chapters/two.wit

+#bibliography:
  @reference_entry
    author: "Berger, J."
    year: 1972
    title: Ways of Seeing
    publisher: Penguin
  reference_entry@
!!

~ master.wit

reference ./chapters/one.wit
reference ./chapters/two.wit

@bibliography
```

Both chapters contribute entries; the master file renders the merged list. Add a chapter, register a `+#bibliography`, and the list grows.

Design principles

1. **Prose unless marked.** The default is writing. Structure is always deliberate and always visible. Nothing the writer types means anything special by accident.
2. **One source, many outputs.** The document is a tree. The renderer decides what the tree becomes.
3. **Cost scales with rarity.** The thing you do most — write sentences — costs nothing. The rare things carry a light, opt-in mark.
4. **Two hats, never at once.** The prose writer never has to think like a programmer. The programmer never has to disturb the prose.

5. **Name the meaning.** Node names carry intent, so the source reads as the thing it is — `@keeper` reads as “the keeper speaks,” `@finding` as “this is a finding,” `@hero` as “this is the hero section.”
6. **Defer to the renderer.** Wit expresses what the writer means. The system downstream supplies what that means in context.

Syntax reference

Syntax	Meaning
`_x_`	italic
`*x*`	bold
`~ x`	line comment (to end-of-line)
`~~ x ~~/`	block comment (inline or multi-line)
`@name`	use a node / bare reference
`@name ... name@`	use a node with a body
`@name(k v, k v)`	parens-form call (space separator)
`@name(k: v, k: v)`	parens-form call (colon separator)
`@name |k v|`	pipe-form param (space separator)
`@name |k – v|`	pipe-form param (multi-word key)
`@name |flag!|`	pipe-form flag
`@name { k – v, k – v }`	record-arg call (self-closing)
`@name\n  k: v\nname@`	form-fill body
`@name k:v k:v name@`	body-scatter (zero-whitespace `id:value`)
`@x.field.0`	dotted data access
`#name ... name#`	block definition
`#name: value !!`	single-line definition
`#name:` … `!!`	multi-line value definition
`+#name`	additive partial — merges across declarations
`||a, b||`	capture parameters (in a definition)
`::name::`	interpolate a captured parameter
`...`	body slot (where invocation content renders)
`! ... !`	value-block inside a collection cell
`"..."`	quoted string (commas, whitespace, newlines)
`{ k – v, k: v }`	record
`[ a, b, c ]`	collection
`(if cond)` `(else)` `(end)`	conditional

Syntax	Meaning
`(each @x as i) (end)`	iteration
`reference ./path`	pull in definitions from another file
`<% ... %>`	JavaScript block / inline expression
`@scriptCall(fn, ...)`	invoke a script-defined function

Wit. Your intent, made manifest.